VODAFONE · TOURIST PACK PLATFORM

Technical Documentation Frontend & Backend

A complete reference for every page, service, integration, and data flow across the Next.js frontend (`tourists-pack`) and Spring Boot backend (`vodafone-project-backend`).

Scope: Pages & UX flows, PayPal payment architecture, passport/ID scanning, 3D map rendering, Daily Drop game engine, discount logic, database schema, security model.

Frontend stack: Next.js 15.2.4 (App Router, Turbopack), React 19, TypeScript 5

Backend stack: Spring Boot 4.1.0, Java 21, PostgreSQL (Supabase)

Generated: September 2026

Table of Contents

1. Platform Overview

1.1 Architecture at a glance

The platform is two independently-run applications with no shared code or monorepo tooling — they only ever talk over plain HTTP.

Layer	Technology	Port	Responsibility
Frontend	Next.js 15.2.4 (App Router, Turbopack), React 19, TypeScript	3000	All UI, plus a thin server-side API layer (`app/api/paypal/*`) that keeps PayPal secrets off the browser
Backend	Spring Boot 4.1.0, Java 21	8080	Business logic, PostgreSQL persistence, PassKit & email integrations
Database	PostgreSQL, hosted on Supabase	—	Twelve entities; schema kept in sync automatically via Hibernate `ddl-auto=update` (no Flyway/Liquibase)

1.2 How the two apps communicate

There is no API client library and no generated types — every TypeScript interface describing a backend DTO is hand-maintained in `features/activation/types/tourist.ts` to match the Java side.

> **TWO-LAYER PAYPAL ARCHITECTURE**
>
> PayPal calls never go directly from the browser to the Spring Boot backend. The browser talks to Next.js API routes, which hold the PayPal client secret server-side, talk to PayPal, and only then call the backend with proof of payment. This is covered fully in §3.1.

> **NO AUTHENTICATION LAYER, ANYWHERE**
>
> Neither app has a login flow. Identity in the game hub and subscription-status pages is carried entirely by `touristId` , a UUID passed as a URL query parameter. The backend has no Spring Security dependency at all and accepts this UUID at face value on every endpoint that takes it. This is a deliberate simplicity tradeoff, and its implications are documented fully in §7.1.

2. Frontend — Pages

Organised by feature, not file type, under `app/` (routes) and `features/` (feature modules). All pages below are client components (`"use client"`) unless noted.

2.1 Home — `app/page.tsx`

`app/page.tsx`

The largest page in the app, built from several independently-animated sections rendered top to bottom:

Promo banner

A full-width button that smooth-scrolls to `#packages` on click. Visually it reads "Summer Offer • Guaranteed Reward with every activation" — the middle glyph is a plain `•` bullet separator, purely decorative.

Hero banner & the 3D Albania shape

Full-bleed background photo with a stacked red/white headline and an interactive rotating outline of Albania. The mechanics of this effect are covered in full in §3.3 — in short, it is **not** a 3D engine or map library, it is a CSS `perspective` + `transform-style: preserve-3d` trick applied to a masked video and an SVG outline, tilted on mouse movement.

Pack Finder quiz

A five-step client-side wizard (`id="pack-finder"`) driven by two pieces of state: `quizStep` (1–5) and `quizAnswers` (`destination`, `duration`, `primaryNeed`, `regionalTravel`). Each answer is applied by `handleAnswer(field, value)`, which merges it into `quizAnswers` and advances the step. On the final step, a `recommendedPack` is chosen entirely client-side from whatever packs already loaded, using simple rules:

- `regionalTravel === "Yes"` or a 22–30 day trip → the longest/priciest available pack
- 16–21 days → the second pack in the list
- otherwise → the first (cheapest/shortest) pack

> **NOT A BACKEND FEATURE**
>
> This recommendation is index-based against the already-fetched `packs` array — there is no dedicated backend "recommend a pack" endpoint. If pack ordering changes on the backend, the quiz's recommendation logic changes with it.

All Available Packs grid

Fetched once on mount via `GET` `{NEXT_PUBLIC_API_BASE_URL}/packs` with `cache: "no-store"`, showing three skeleton placeholders while loading. The first three packs returned are labelled *starter*, *popular*, and *best-value* purely by their array index (0/1/2) — this is a presentational badge, not a flag stored on the `Pack` entity.

Custom Plan Builder

Embedded section (`id="custom-plan"`) delegating to `CustomPlanBuilder.tsx` — a slider-driven pricing tool that debounces changes (300ms) and calls the backend's live quote endpoint on every change (see §4.3).

Digital Pass section

The Apple/Google Wallet buttons shown here on the homepage are **decorative only**. Clicking either simply pops an alert ("You must activate a package first!") and scrolls back up to the packages grid — no wallet action is actually triggered from this page. Real wallet issuance only ever happens after a paid activation (§5.1, step 9).

2.2 Activate — `app/activate/page.tsx`

`app/activate/page.tsx`

A single-step tourist-details form. Reads `packId` from the URL via `useSearchParams()` (wrapped in `<Suspense>`, required by the App Router for that hook) and fetches the pack's full details on mount via `GET /packs/{packId}`.

On submit:

1. A fresh `orderId` is generated client-side (`crypto.randomUUID()` , with a timestamp+random fallback for older browsers).
2. The tourist's form data (name, email, passport number) is written into `sessionStorage` under that order id — never into the URL. See §3.6 for exactly why.
3. The browser is routed to `/payment?orderId=...&packId=...` — only opaque identifiers travel in the URL, never PII.

2.3 Payment — `app/payment/page.tsx`

`app/payment/page.tsx`

The most complex page in the app, with two states: `PAYMENT` and `SUCCESS` .

On mount

- Reads the tourist's form data back out of `sessionStorage` by `orderId` . A miss here (bookmarked link, a fresh tab, private browsing) is treated as an expected "your session has expired, please start again" state — not an error path.
- Fetches a PayPal client token: `GET` `/api/paypal/client-token` , used to power the hosted card-fields flow.
- Re-fetches the pack's price fresh from the backend: `GET` `/packs/{packId}` .

Creating the order

`createOrder()` calls `POST` `/api/paypal/create-order` with just `{ packId, email }`. This Next.js route (not the browser) is where the real price is computed, the returning-tourist discount is looked up, and the Lek→EUR conversion happens — full trace in §3.1.

Capturing payment

`captureOrder(orderID, actions)` calls `POST` `/api/paypal/capture-order` with `{ orderID, packId, formData }`. This is the point at which money actually moves and the backend activation record is created — see the full trace in §3.1 and §5.1. On success the response's `orderRef`, `touristId`, and `transactionId` are stored, the sessionStorage entry is cleared, and the page switches to `SUCCESS`.

If PayPal reports an `INSTRUMENT_DECLINED` issue, the code calls `actions.restart()` so the buyer can pick a different funding source instead of the flow dead-ending.

Payment methods offered

Two coexist inside one shared `<PayPalScriptProvider>`: `<PayPalButtons fundingSource={FUNDING.PAYPAL}>` (deliberately restricted to the PayPal-branded button only, so PayPal's own generic unstyled "Debit or Credit Card" fallback button never appears), and a custom themed `CardPaymentForm` built on PayPal's hosted Card Fields.

On success

Shows `WalletSyncStatus` (a short scripted progress list — "Check your inbox," "One more thing: play a quick game next"), then navigates to `/game-hub?touristId={touristId}`. **This is the only place in the entire frontend where `touristId` is minted into a URL.**

2.4 Game Hub — `app/game-hub/page.tsx`

`app/game-hub/page.tsx`

Reads `touristId` from the URL query string. If it's missing, the page shows a "couldn't find your session" message and makes no API calls at all — there's no fallback lookup by any other means. If present, everything is wrapped in `<GameProvider touristId={touristId}>` (the game state context, §3.4), rendering the `DailyDropCard` scratch-card flow, a `TodaysPrizes` odds table, an FAQ accordion, and a `RewardPassReveal` overlay when a play results in a win.

2.5 My Pack — `app/my-pack/page.tsx`

`app/my-pack/page.tsx`

The tourist's data-usage / subscription-status dashboard. Reads `touristId` from the URL and calls `GET /tourists/{touristId}/subscription`. Renders:

- A status badge (`ACTIVE` / `PENDING` / `CANCELLED` / `FAILED`) with a colour keyed off the status value
- A computed "days remaining" banner, derived client-side from the subscription's expiry timestamp
- Pack spec rows (data allowance, minutes, validity)
- Key dates, formatted via `formatDate()` — returns `"N/A"` for a missing date rather than leaving it blank
- Order & payment reference info, current game credit balance
- An "Add to Apple Wallet" link built directly from the backend-supplied `appleWalletUrl` — never constructed client-side

---

2.6 Privacy & Terms — `app/privacy/page.tsx`, `app/terms/page.tsx`

`app/privacy/page.tsx · app/terms/page.tsx`

Static legal content, no client state, no API calls. The privacy page's passport-scanning section describes, in plain language, exactly what the on-device OCR flow does and does not do — and it is factually accurate against the real implementation traced in §3.2: the photo never leaves the device, only three fields are ever extracted, and everything is discarded the instant scanning finishes. It also explicitly names the four third parties that do receive data: **PayPal** (payment), **PassKit** (wallet pass issuance), **Supabase** (database host), and **Gmail/Google** (confirmation email delivery) — see §7.5 for the full data inventory.

2.7 Wallet fallback pages — `app/wallet/apple/[id]`, `app/wallet/google/[id]`

`app/wallet/apple/[id]/page.tsx · app/wallet/google/[id]/page.tsx`

Trivial dynamic routes that read the subscription id from the path and render `WalletUnavailableNotice`. These are reached **only** as a fallback, when PassKit wallet enrollment failed server-side during activation (§5.5) — they are not the normal "add to wallet" path, which instead links straight to PassKit's own hosted pass URLs.

3. Frontend — Feature Deep Dives

3.1 PayPal Payment Integration

`features/activation/lib/paypal.ts · features/activation/lib/currency.ts · app/api/paypal/{create-order,capture-order,client-token}/route.ts`

Why a Next.js API layer sits in front of PayPal

The browser never talks to PayPal's REST API directly for order creation or capture. `getPayPalAccessToken()` performs an OAuth2 client-credentials exchange against PayPal using `PAYPAL_CLIENT_SECRET` — an environment variable deliberately **without** the `NEXT_PUBLIC_` prefix, so Next.js never bundles it into client-side JavaScript. This is the entire reason the intermediate API layer exists.

Currency conversion

Pack prices are stored in Albanian Lek. `allToEur()` divides by a fixed rate of **100 ALL per EUR** and rounds to the nearest whole Euro — a deliberately clean marketing rate rather than a live market rate (real ALL/EUR hovers around 95–100), so a 2700 ALL pack becomes a flat €27.

create-order route — full flow

1. Rate-limited per client IP (10 requests/minute).
2. Re-fetches the pack fresh from the backend by id — the price a client supplied is never trusted.
3. If an email was supplied, calls `GET` `/tourists/discount-by-email` on the backend. If eligible, captures `discountPercent` from the response.
4. Converts the price to EUR via `allToEur()`, then applies the discount: `amount = amount × (1 - discountPercent / 100)`.
5. Creates the real PayPal order (`POST /v2/checkout/orders` , `intent: "CAPTURE"`) carrying `custom_id: packId` and the final discounted EUR amount.
6. Returns `{ orderID, discountApplied }` to the browser.

capture-order route — full flow

Rate-limited tightest of all three routes (5/minute — "moves real money").

1. Calls PayPal's capture endpoint — this is the step where money actually moves (create-order only authorizes).
2. On a declined instrument, surfaces the specific PayPal issue code back to the browser so it can offer `actions.restart()`.
3. Calls the backend: `POST` `/api/v1/activations` with the pack id, the tourist's form fields, the PayPal order id and capture id, the amount paid, and currency. **This backend call is what actually creates the activation record** — it never happens from the browser.

4. If the backend call fails *after* PayPal capture already succeeded, a `CRITICAL:` log line is written (money taken, no activation recorded) and the user is shown a 502 asking them to contact support with the capture id as a reference — it deliberately does not pretend success.

React SDK usage

`@paypal/react-paypal-js` (v10.3.0) wraps the payment UI in `<PayPalScriptProvider options={{ clientId, currency: "EUR", intent: "capture", components: "buttons,card-fields", dataClientToken }}>`. The card form uses `<PayPalCardFieldsProvider>` with individual hosted `PayPalNameField` / `NumberField` / `ExpiryField` / `CVVField` components and the `usePayPalCardFields()` hook — the raw card number is handled entirely inside PayPal's hosted iframes and never touches this application's JavaScript.

3.2 Passport / ID Scanning

`features/activation/lib/passportScan.ts · cameraCapture.ts · components/PassportScanButton.tsx, PassportLiveCapture.tsx, PassportCaptureGuide.tsx`

> **CONFIRMED BY CODE TRACE**
>
> No image data ever leaves the browser. `scanPassport()` and `scanPassportFromCapture()` operate purely on in-memory `<canvas>` / `<img>` elements and Tesseract's local WASM worker — there is no `fetch` / `XHR` call anywhere in the scanning code path.

Why the MRZ, not the whole document

Every passport and most national ID cards carry a standardized **Machine Readable Zone** — the monospace strip of text at the bottom, the same field every airport e-gate reads. It's self-validating via built-in check digits, which lets a bad OCR read be rejected outright instead of silently corrupting a legal field like a passport number.

Two capture paths

Chosen by `isCameraCaptureSupported()`, which requires `window.isSecureContext` (true on HTTPS, or exactly `localhost` — false on a plain-HTTP LAN IP):

1. Live camera (preferred)

`PassportLiveCapture.tsx` opens `getUserMedia` with a rear-camera preference, shows a live video feed, and overlays a guide band covering the bottom 30% of the frame height with a 6% horizontal margin — the tourist visually aligns the MRZ strip into this box. On capture, the guide rectangle is mapped back to native video pixel coordinates and two canvases are produced: a tightly-cropped, upscaled `primary` region and a `fallback` full frame.

2. File picker (fallback)

A native `<input type="file" accept="image/*" capture="environment">` opens the device's own camera app. Since a plain file input can't show a live overlay, `PassportCaptureGuide.tsx` shows a one-time static diagram of correct framing immediately beforehand. The captured image is then cropped to its bottom 30% and OCR'd, with the full original image kept as a fallback source.

OCR & parsing pipeline

Step	Detail
Engine	`tesseract.js` (v7.0.0) — `createWorker("eng")`, character whitelist restricted to `A–Z 0–9 <`
First pass	`PSM.SINGLE_BLOCK` mode on the tightly-cropped `primary` source — correct for a clean MRZ crop
Line extraction	Regex `/^[A-Z0-9<]{25,46}$/` — real MRZ lines are 30/36/44 characters (TD1/TD2/TD3), with slack allowed for OCR noise. Takes the last 3 matching lines if found (ID card format) else the last 2 (passport format)
Retry pass	If fewer than 2 valid lines are found and a fallback source exists, retries with `PSM.SPARSE_TEXT` — suited to a full document photo with scattered text and a face
Parsing	`mrz` package (v5.0.2), `parseMrz(lines, {"{"} autocorrect: true {"}"})` — autocorrect fixes common OCR ambiguities (O/0, I/1, S/5) in fields known to be purely numeric or alphabetic
Data extracted	Only `firstName`, `lastName`, `documentNumber` — even though the MRZ also encodes date of birth, sex, nationality, and expiry date, those fields are parsed internally by the library and immediately discarded, never surfaced or stored

Names are title-cased and MRZ filler characters (`<`) are replaced with spaces; the passport number has all `<` characters stripped. Every canvas is explicitly zeroed out (`releaseSource()`) in a `finally` block so no image data outlives the scan.

3.3 3D Rotating Map & Store Locator

`app/page.tsx (hero section) · app/styles.css · features/stores/components/StoreMapPins.tsx · features/stores/utils/geo.ts`

> **NOT A 3D ENGINE, NOT WEBGL, NOT A REAL MAP**
>
> The "3D rotating Albania" effect is a CSS `perspective` + `transform-style: preserve-3d` trick applied to a flat 2D shape — there is no Three.js, no Mapbox, no real 3D geometry anywhere in this codebase.

How it's actually built

- `.hero-3d-stage` sets `perspective: 1400px` — the virtual "camera" distance for the effect.
- `.hero-map-3d` is the element that rotates: `transform-style: preserve-3d` plus an idle CSS keyframe animation (9-second loop) gently oscillating between `rotateY(-10deg)` / `rotateY(10deg)` and a matching

X-axis tilt.

- Inside it, a looping muted autoplay `<video>` of Albania (hosted on Supabase Storage) is CSS-**masked** into the country's exact silhouette using `-webkit-mask-image: url('/assets/albaniaMap.svg')`.
- A second copy of the same SVG is drawn on top as a crisp outline, recoloured red via a CSS `filter` chain (`brightness` + `sepia` + `saturate` + `hue-rotate`), plus a radial-gradient tint layer for edge glow.

Mouse-follow tilt

On `mousemove` over the hero stage, the cursor's position relative to the stage is computed as two values in the range [−0.5, 0.5]. These feed directly into an inline `transform: rotateY(relX × 34deg) rotateX(relY × -22deg)` set via a plain DOM ref mutation (`map.style.transform = ...`) — deliberately **not** React state, so the browser never re-renders on every mouse-move event. An `is-interacting` class is added while hovering, which disables the idle CSS animation and speeds up the transition. On mouse-leave, both the class and the inline transform are cleared, letting the idle animation resume. **Framer Motion is not involved in this effect at all** — it's hand-rolled vanilla CSS and refs; Framer Motion is used elsewhere (the game-hub reveal modal, wallet sync steps) instead.

Store locator pins

`StoreMapPins.tsx` overlays six Vodafone store markers (Shkodër, Shengjin, Tiranë, Durrës, Vlorë, Sarandë) on top of the shape. Positions are **hardcoded percentage offsets** per store — not computed from latitude/longitude, since the SVG itself is drawn at an artistic angle rather than a true map projection. Each pin's pulse animation is staggered by a per-index delay so they don't blink in sync. Clicking a pin opens a popup with the store name, a coverage note, and a "View on Google Maps" link built from a hardcoded URL per store.

`geo.ts` exports a `googleMapsUrl()` helper for building a Maps link from coordinates or a place id, but it is not actually called anywhere in the codebase — every store's Maps link is a hardcoded string instead.

3.4 Game Hub Scratch Card Engine

```
features/game-hub/components/ScratchCard.tsx, DailyDropCard.tsx, RewardPassReveal.tsx · context/GameContext.tsx
```

GameContext — the credit state source of truth

A React Context holding `touristName`, `credits`, `hasPlayedToday`, `dailyClaimAvailable`, `nextClaimAt`, `loading`, and `error`. Three functions mutate/refresh it:

Function	Call	Behaviour
`refresh()`	`GET /game-hub/state?touristId=...`	Runs automatically on mount and whenever `touristId` changes
`claimDailyCredit()`	`POST /game-hub/claim-daily-credit`	Re-applies the returned state after claiming
`playDrop()`	`POST /drop/play`	Returns `{ won, prize }`; always calls `refresh()` in a `finally` block regardless of outcome

A derived value, `canPlay = credits > 0 && !hasPlayedToday`, gates the UI. Consumed everywhere via a `useGame()` hook that throws if used outside the provider.

The scratch-off mechanic — real canvas drawing, not an image swap

1. On mount, the **actual prize face is drawn first** — "YOU WON" plus the prize label and sponsor, or a "try again tomorrow" message on a loss. The true outcome is baked into the canvas from the very start.
2. A second, fully opaque foil layer is painted on top — a diagonal red gradient with "SCRATCH HERE" text — completely hiding the prize face.
3. On pointer drag, each stroke uses `ctx.globalCompositeOperation = "destination-out"` to punch a small circle out of the foil layer — this genuinely **erases** pixels rather than painting a new colour, exposing the real prize drawn underneath in step 1.
4. Every stroke samples the canvas's alpha channel to estimate the cleared-area ratio. Once it crosses a 45% threshold, a reveal callback fires exactly once.

> **OUTCOME IS DECIDED BEFORE SCRATCHING STARTS**
>
> The random prize draw happens server-side the instant the tourist taps "Reveal today's drop" — before any scratching occurs. That's why the canvas can legitimately draw the real prize underneath from frame one: the server has already decided the outcome by the time the card is even shown.

Daily flow orchestration

`DailyDropCard.tsx` branches the UI on state: claim available → "Claim daily credit" button; can play → "Reveal today's drop" button (which calls `playDrop()` and mounts the scratch card); already played today → a "come back tomorrow" message; out of credits → a message plus a live countdown to the next credit. A documented ordering subtlety in the code: the "currently revealing" state must be checked *before* "already played today" in the render logic, because the background refresh from `playDrop()` flips "played today" to true immediately (it's recorded server-side before scratching even finishes) — without that check, the "claimed" UI would flash in front of the scratch card before the tourist ever sees it.

3.5 Icon System & UI Libraries

`shared/components/icons.tsx`

Four icons are hand-drawn as custom SVGs, explicitly to avoid the generic "templated" look of stock outline icons standing in for concepts that deserve a real, specific glyph:

- **SignalBarsIcon** — genuine ascending signal bars, for coverage/data indicators
- **NfcWaveIcon** — concentric arcs radiating from a point, an actual NFC "tap" glyph rather than a rotated Wi-Fi icon
- **SpeechMarkIcon** — a Vodafone-style speech-mark brand accent
- **BadgeCheckIcon** — a heavier, more confident checkmark badge than a thin outline check, used for trust/success moments

Every other icon in the app comes from **lucide-react** (v0.525.0) — dozens of usages across the codebase (`MapPin`, `Calendar`, `ScanLine`, `Lock`, `Gift`, `Trophy`, `Wallet`, `ShieldCheck`, and many more) — plus two one-off inline brand SVGs (Apple and Google wordmark glyphs) kept local to `app/page.tsx` since they're single-use wallet-button logos, not shared icons.

Other notable dependencies

Package	Version	Role
`framer-motion`	^13.1.0	Animated transitions: game-hub reveal modal, wallet sync progress steps, scratch-card entry
`next-themes`	^0.4.6	Light/dark theme switching via `ThemeProvider`, `data-theme` attribute strategy
`class-variance-authority`, `clsx`, `tailwind-merge`	—	Class-name composition utilities, unified in a single `cn()` helper
`@fontsource/manrope`, `@fontsource/archivo`, `@fontsource/ubuntu`	^5.3.0	Self-hosted static font files — replaced `next/font/google` and a Google Fonts CSS `@import` to remove a runtime network dependency on Google's font CDN

3.6 Checkout Session Handoff

`features/activation/lib/checkoutSession.ts`

Tourist PII (name, email, passport number) moves between the Activate and Payment pages via `sessionStorage`, keyed as `"checkout:" + orderId`. The reasoning, born out in the code:

- **Not URL parameters**: a query string containing a passport number and email would land in browser history, server access logs, and any outgoing `Referer` header — an unacceptable exposure for data this sensitive, especially since the payment page can independently re-derive the price from just a `packId`.

- **Not** `localStorage`: shared indefinitely across every tab on the origin and never expires on its own — the wrong lifetime for data that's only needed for a few minutes, once.
- `sessionStorage`: scoped to a single tab, cleared automatically when that tab closes — matching the actual need exactly. The known tradeoff is explicit in the code: it survives a client-side navigation or a full reload within the same tab, but is gone on a bookmarked direct visit, a new tab, or a reopened closed tab. The payment page treats that as an expected "session expired" state, never a silent error.

Both the save and read paths wrap their `sessionStorage` calls in `try/catch`, since strict private-browsing modes can throw on storage access.

3.7 `touristId` on the Frontend

Traced every read of this identifier across the frontend:

- Minted exactly once — by the backend's activation response, surfaced in `app/payment/page.tsx` — and immediately used to build the `/game-hub?touristId=...` redirect URL.
- Read via `useSearchParams().get("touristId")` in both the Game Hub and My Pack pages: a raw URL query parameter, with **no cookie, JWT, or Authorization header involved anywhere** in the API helper functions that use it.
- Every game-hub/my-pack API call sends this UUID as a plain query parameter or JSON field, and the backend accepts it at face value (§7.1 covers the full implication of this).

3.8 Edge-Route Rate Limiting

`features/activation/lib/rateLimit.ts`

A hand-rolled in-memory token-bucket limiter, keyed per client IP (read from the `x-forwarded-for` header). Explicitly documented in its own code as **not safe for multi-instance deployments** — each server instance would count independently, so scaling horizontally would need a shared store like Redis. Three named rules apply to the three PayPal edge routes:

Route	Limit	Rationale
create-order	10 / minute / IP	Guards against hammering the backend and PayPal's own quota
capture-order	5 / minute / IP	Tightest of the three — this is the step that moves real money
client-token	15 / minute / IP	Most permissive — unauthenticated, called on every checkout page load

4. Backend — API Surface

Base path for all endpoints: `/api/v1`. No endpoint anywhere in the backend requires authentication — there is no Spring Security dependency in the project at all (confirmed against `pom.xml`). Every request instead passes through `RateLimitFilter` (§7.3) and the CORS policy (§7.2).

4.1 ActivationController

Method	Path	Body	Notes
POST	`/activations`	`ActivationRequest`	The single write endpoint for the whole checkout flow — delegates entirely to `ActivationService.activate()`, fully traced in §5.1. Only ever called server-side, by the Next.js `capture-order` route.

4.2 PackController

Method	Path	Notes
GET	`/packs`	All active packs
GET	`/packs/{packId}`	Single pack by UUID
GET	`/packs/sponsors`	Active sponsor/partner offers — a naming quirk (it's about partners, not packs), not a bug

4.3 CustomPlanController

Method	Path	Notes
POST	`/packs/custom/quote`	Pure price calculation from a slider-driven spec (data, minutes, duration) — no database write. Hit on every slider drag, ~300ms debounce on the frontend.
POST	`/packs/custom/build`	Same request shape, but **persists** a real `Pack` row with `isActive=false`, then returns it so the frontend can carry the new pack's id into `/activate` like any other pack.

4.4 GameController

Method	Path	Notes
GET	`/game-hub/state?touristId=`	Full current state: credits, claim availability, played-today flag
GET	`/game-hub/prizes`	Public prize catalogue with display odds — no `touristId` needed
POST	`/game-hub/claim-daily-credit`	Grants the free daily credit, idempotent per day (§5.3)
POST	`/drop/play`	The free daily scratch-card play; spends one credit
POST	`/drop/redrop/play`	A second, independently-tracked daily play — does not spend a credit on the backend side (the frontend must gate this behind its own flow)

Client IP and User-Agent are captured on every play for an audit trail on the `GamePlay` row — not used for any rate-limiting or fraud logic.

4.5 TouristController

Method	Path	Notes
GET	`/tourists/discount-by-email?email=`	Validated (`@Email`) query param. Full logic traced in §5.2.
GET	`/tourists/{touristId}/subscription`	Backs the My Pack page; returns 404 if not found

5. Backend — Core Services

5.1 ActivationService.activate() — the full transactional flow

`service/ActivationService.java`

The entire method runs inside a single `@Transactional` boundary — every step below commits or rolls back together, **except** the email send and PassKit enrollment, which each catch their own failures internally so a paid activation is never rolled back over a downstream integration outage.

1. **Pack lookup.** Loads the `Pack` by id; throws if not found (mapped to HTTP 400).
2. **Duplicate-payment guard.** Looks up an existing `PaymentTransaction` by PayPal order id. If one already exists, throws immediately (HTTP 409) — before touching the tourist or subscription tables at all, so a retried or double-clicked capture is a true no-op rather than a partial write.
3. **Tourist upsert by email.** Looks up a `Tourist` by the submitted email. If found, the *same row and UUID* are reused, and its name/passport fields are overwritten with whatever was just submitted. If not found, a brand-new `Tourist` with a fresh UUID is created. This single mechanism is the entire reason returning tourists keep their UUID across visits — matched purely by the unique `email` column.
4. **Discount consumption.** If the tourist has an unredeemed pack-discount prize, its `redeemedAt` timestamp is stamped right here. By this point the discount has already been baked into the price PayPal charged (the frontend's create-order route applied it before this endpoint was ever called) — this step exists purely to close the loop so the same discount can't be reused on a future purchase.
5. **Order reference generation.** `"VF-" +` six uppercase hex characters taken from a fresh random UUID. Not checked for collisions against existing references — relies solely on a database `unique` constraint as a backstop (astronomically unlikely to collide at this data volume, but a genuine, if extremely rare, 409 is possible).
6. **UserSubscription creation.** New row with the pack, tourist, order reference, chosen delivery method, status set directly to `ACTIVE` (there's no PENDING/PROCESSING intermediate stage used in practice, despite those enum values existing), and an `activatedAt` timestamp. If delivery is eSIM, provisioning happens synchronously here and the activation code / QR install link / phone number are stamped straight onto the subscription.
7. **PaymentTransaction recorded.** One row capturing the PayPal order id, capture id, amount paid, currency, and a `COMPLETED` status — the durable proof-of-payment record this platform didn't originally have.
8. **Credit grant.** A `CreditTransaction` with `delta: +1` and reason `PACK_ACTIVATION_BONUS`, linked to both the tourist and the subscription — a pure ledger insert, no balance column is touched anywhere in the schema.
9. **PassKit enrollment.** Local fallback wallet URLs are pre-computed first (pointing at the frontend's own wallet fallback pages). PassKit is then called; if it succeeds, the subscription is updated with the real PassKit member id and pass URL, overwriting the fallback links. If it fails, failure is fully swallowed inside the PassKit service (§5.5) — activation always succeeds regardless of PassKit's availability.

10. **Welcome email.** Sent synchronously, inline, before the method returns — but internally try/caught, so an SMTP failure is logged and never propagated to the tourist's HTTP response.
11. **Response.** Returns the subscription id, tourist id, payment id, order reference, status, and the eSIM install-link URL (not a raw QR image — any page wanting to show a QR renders one from this URL itself).

5.2 Returning-Tourist Discount Engine

Eligibility is entirely **derived** — there is no separate "returning tourist" flag stored anywhere. A tourist qualifies for a discount if, and only if:

1. They already exist as a `Tourist` row, matched by email, **and**
2. They have, at some point, played the Daily Drop game and won a prize of type `PACK_DISCOUNT` (as opposed to `PARTNER_OFFER`) that has never been redeemed (`redeemedAt IS NULL`).

The lookup query is deliberately written as **native SQL** rather than JPQL. The reason, per an in-code comment: comparing the `prize_type` enum column via JPQL causes Hibernate to cast the literal to a Postgres type name derived from the Java enum class, which doesn't match the actual native Postgres enum type on that column — producing an "operator does not exist" error. Casting both sides to `::text` sidesteps the mismatch entirely.

```
SELECT gp.* FROM game_plays gp
JOIN prizes pz ON pz.id = gp.prize_id
WHERE gp.tourist_id = :touristId
  AND pz.prize_type::text = 'PACK_DISCOUNT'
  AND gp.redeemed_at IS NULL
ORDER BY gp.played_at DESC
LIMIT 1
```

The discount amount is not a fixed, platform-wide percentage. It's read directly off `Prize.discountPercent` — a nullable column on the `prizes` table — meaning different seeded prize rows can carry different discount percentages. Which prize (and therefore which percentage, if any) a tourist wins is entirely down to the weighted random draw described in §5.3; the discount value itself is configured data, not application logic.

The lookup endpoint (`GET /tourists/discount-by-email`) is a pure, read-only query — it never consumes the discount itself. Consumption only happens inside `ActivationService.activate()`, step 4 above, at the moment a purchase is actually completed.

5.3 GameService

`service/GameService.java`

Timezone

Configurable via `app.game-hub.timezone`, defaulting to `Europe/Tirane` if unset. Every notion of "today" in daily-claim and daily-play logic is computed against this zone.

Daily credit claim — true idempotency

An application-level check (does a claim already exist for this tourist today?) runs first for a fast, friendly error, but the **real** guarantee is a database-level unique constraint on `(tourist_id, claim_date)` on the `DailyCreditClaim` table. The insert is executed with an immediate flush specifically so that constraint violation surfaces within the same request (as HTTP 409) rather than being silently lost — this closes the race window that the application-level check alone would leave open under concurrent requests (a double-tap, two open tabs, a retried request).

Daily play idempotency

Enforced the same way, via a separate unique constraint on `(tourist_id, game_id, played_date, drop_type)` on the `GamePlay` table. Because `drop_type` is part of that key, the free daily drop and a paid redrop are tracked as fully independent daily limits — a tourist can play both once each on the same day.

Weighted prize selection

Loads every *active* prize. Each prize's effective weight treats a missing weight as zero and clamps any negative weight to zero. If every active prize has a zero/null weight, the draw falls back to a uniform random pick across all of them. Otherwise, a classic weighted-bucket draw is used: roll a random number under the total weight, then walk the prize list accumulating weight until the roll falls inside a prize's bucket. The same effective-weight math powers the public odds shown on the `TodaysPrizes` component, so the displayed chance percentages are always mathematically consistent with the real draw.

Credit ledger, confirmed

There is no stored balance column anywhere on the `Tourist` entity or elsewhere. The balance is a live aggregate — the sum of every `CreditTransaction.delta` row for that tourist, recomputed on every read. Every credit-affecting action (pack activation bonus, daily claim, spending a play) is simply a new ledger row.

Play flow

Shared logic for both the free drop and the paid redrop: verify the game exists, verify it hasn't already been played today for this drop type, and — *only for the free drop* — verify a positive credit balance and debit one credit. A paid redrop does not spend a credit on the backend side at all, implying any gating for that flow (payment, an ad view, etc.) lives entirely on the frontend/elsewhere. A random prize code is generated, the weighted draw runs, and a `GamePlay` row is saved with `won` hardcoded to true — there is no server-side "loss" outcome in this logic today, despite the frontend having a "So close" no-win UI state prepared for one.

5.4 EmailService

`service/EmailService.java`

Standard `JavaMailSender` over Gmail SMTP (port 587, STARTTLS). Confirmed contents of the welcome email, sent after every successful activation:

- Subject including the order reference
- An inline embedded header image and an order summary (pack title, data/minutes/duration)
- A delivery section that branches on method: for eSIM, a **QR code generated server-side** (via ZXing, embedded as an inline image, not merely linked) encoding the activation code, an "Add eSIM to iPhone" button, and a monospace manual-entry code; for a physical SIM, pickup instructions
- Action buttons: "Open Game Hub" and "Check Your Data Usage," plus "Add to Apple Wallet" / "Add to Google Wallet"
- A short FAQ accordion and a footer

The send is fully **synchronous** — it happens inline inside the activation transaction, and the tourist's HTTP response waits on the SMTP round-trip. Any failure (auth, connection, or send-time) is caught and only logged, never re-thrown, specifically so a tourist who has already paid never has their activation fail purely because of a mail server issue.

5.5 PassKitService

`service/PassKitService.java`

A single enrollment call (`POST /members/member` against PassKit's API) carries the program/tier configuration, the subscription's UUID as PassKit's external id, the tourist's name and email, and a metadata block including the pack's data/minutes/validity, a **fixed calendar expiry date** computed once at enrollment time (not a live countdown — that would require a scheduled job that doesn't exist), and deep links back to the game hub and usage dashboard.

Apple and Google wallet passes are not two separate API calls — one enrollment produces a single pass URL, and the two wallet flavours are simply different file-extension suffixes appended to that same URL (`.pkpass` for Apple, `.gpay` for Google). The pass's own QR/barcode is generated automatically by PassKit from the member id — there is no per-enrollment barcode override available in the API, meaning the frontend must never attempt to render its own substitute QR for this particular pass; PassKit's built-in one is the only valid one.

The entire method is wrapped in a broad try/catch: any failure (network, a bad status code, a malformed response) is logged and the method returns an empty result rather than throwing — enrollment is explicitly best-effort, matching the same philosophy as email delivery, so a PassKit outage never blocks a paid activation from completing.

6. Data Model

6.1 Entity Reference

Entity	Table	Notable columns	Relationships
Tourist	tourists	`email` unique + not null (the upsert key)	Root of every other entity below
UserSubscription	user_subscriptions	`order_ref` unique; delivery method & status as native Postgres enums; nullable eSIM fields	→ Tourist, → Pack
Pack	packs	price in Lek (DECIMAL 10,2); `roamingDetails` stored as JSONB but typed as a plain String; `isCustom` deliberately a boxed `Boolean` (not primitive) to tolerate legacy null rows under `ddl-auto=update`	→ many PackFeature
PackFeature	pack_features	label, icon key (maps to a lucide-react icon name on the frontend), sort order	→ Pack
PaymentTransaction	payment_transactions	`paypal_order_id` unique + not null — the duplicate-payment guard key; `paypal_capture_id` unique but nullable	→ UserSubscription, → Tourist
CreditTransaction	credit_transactions	signed `delta` int; reason enum	→ Tourist (required), → UserSubscription (nullable — daily claims aren't tied to a subscription)
DailyCreditClaim	daily_credit_claims	unique constraint on (tourist, claim date) — the real idempotency enforcement	→ Tourist
GamePlay	game_plays	unique constraint on (tourist, game, played date, drop type); IP address + User-Agent recorded as an audit trail only; nullable `redeemedAt`	→ Prize (nullable), → Tourist, → Game
Prize	prizes	nullable integer weight; active flag; type (partner offer / pack discount); nullable discount percent; **id is not auto-generated** — must be seeded	→ PartnerOffer (nullable)
Game	games	unique `code` (e.g. `"daily_drop"`, looked up by `GameService`)	leaf entity
Partner	partners	name, logo, category, active flag; **id not auto-generated**	leaf entity
PartnerOffer	partner_offers	discount label, active flag, sort order; **id not auto-generated**	→ Partner

6.2 Relationships & Seeding Notes

`Prize`, `Partner`, and `PartnerOffer` have no auto-generated primary key — all three tables are expected to be pre-seeded with fixed UUIDs (via SQL seed scripts or manual inserts) rather than created through any application code path. No controller anywhere exposes a write endpoint for any of these three entities, nor for `Game`.

Beyond the relationships shown above, note that `CreditTransaction` optionally links to a subscription, `GamePlay` links to both a `Game` and (nullably) a `Prize`, and `Prize` in turn optionally links to a `PartnerOffer` — the discount and partner-reward systems share the same prize table, distinguished only by the `prizeType` column.

7. Security & Configuration

7.1 `touristId` Threat Model

> **NO OWNERSHIP CHECK EXISTS**
>
> Every endpoint that accepts a `touristId` — game-hub state, daily claim, drop play, subscription status — accepts it as a plain, unauthenticated UUID. There is no cookie, JWT, session, or `Authorization` header check anywhere in the request path on either the frontend or the backend. Anyone who obtains a valid `touristId` — for example, by observing a URL over someone's shoulder, or from a forwarded activation email containing the game-hub deep link — can view that tourist's subscription and game state, and can claim their daily credit or play their drop on their behalf.

This is a deliberate simplicity tradeoff rather than an oversight: the identifier is a 128-bit UUID, effectively unguessable in practice, and CORS restricts which web origins can call these endpoints from a browser. But CORS provides no protection against a direct API client, and the UUID's secrecy is the *only* control in place — this is security through obscurity of the identifier, not authentication. Any future hardening of this platform should treat this as the first item on the list.

7.2 CORS

A single `WebMvcConfigurer` bean applies to every path under `/api/**`. The allowed-origin property is parsed as a **comma-separated list** (deliberately, per an in-code comment, to let a developer add their own LAN IP for testing from a phone on the same Wi-Fi without a code change) and is read entirely from configuration — defaulting to `http://localhost:3000` in local development. Only `GET` and `POST` are CORS-allowed (no PUT/DELETE/PATCH), and only the `Content-Type` header is permitted — notably, no `Authorization` header is allowed, consistent with there being no auth layer to carry one.

7.3 Rate Limiting

A backend-wide `RateLimitFilter` (built on Bucket4j, in-memory token buckets keyed per client IP) applies to every request under `/api/v1/**`, independently of the Next.js edge-route limiter described in §3.8:

Bucket	Limit	Applies to
ACTIVATION	5 / minute	`POST /activations`
DISCOUNT_LOOKUP	30 / minute	`GET /tourists/discount-by-email` — specifically to blunt email-enumeration attempts
GAME_ACTION	20 / minute	Daily claim + both drop-play endpoints
CUSTOM_PLAN_QUOTE	300 / minute	Deliberately generous — hit on every pricing-slider drag, roughly one call every 300ms
CUSTOM_PLAN_BUILD	10 / minute	A real database write with no payment gate behind it, so tighter than the default but looser than activation
DEFAULT	120 / minute	Everything else: pack listing, sponsor offers, game-hub reads, subscription status

Exceeding a limit returns HTTP 429 with a small JSON error body. Like the frontend's limiter, this is explicitly single-instance-only — buckets live in a process-local map, so scaling the backend to multiple instances would need a shared store (Redis) to keep the limits meaningful.

7.4 Configuration Reference

All configuration is resolved through `application.properties` placeholders backed by a local, gitignored `.env` file.

Property	Purpose
`spring.datasource.url / username / password`	Supabase PostgreSQL connection
`spring.jpa.hibernate.ddl-auto=update`	Automatic schema sync on startup — no migration tool
`app.cors.allowed-origin`	Comma-separated list of allowed frontend origins
`app.game-hub.timezone`	Timezone for "today" in daily claim/play logic (default `Europe/Tirane`)
`app.frontend.base-url`	Used to build every deep link the backend generates: game hub, my-pack, wallet fallback pages
`passkit.api.base-url / api.token / program-id / tier-id`	PassKit wallet issuance credentials & program config
`spring.mail.host / port / username / password`	Gmail SMTP credentials for outbound email

7.5 Data Usage & Privacy Summary

Reflecting both the privacy policy copy and the actual code paths that back it:

Data	Collected how	Shared with
Identity details (name, passport/ID number)	Typed manually, or auto-filled via the on-device MRZ scanner (§3.2) — the source photo never leaves the device	Stored in the platform's own database (Supabase-hosted); required by Albanian telecom law to be recorded against every SIM activation
Email address	Typed on the activation form	Vodafone (confirmation delivery via Gmail SMTP), PassKit (wallet pass), Supabase (storage)
Payment confirmation	PayPal order/capture reference and amount — the card number itself is never seen by this platform	PayPal (processes the payment directly); the reference is stored, not the card details
Pack & usage data	Generated by the activation and game-hub flows	Stored in the platform's database only

Session-scoped checkout data (§3.6) is held only in the browser's own `sessionStorage` and is never transmitted to any server until the tourist submits the activation form — it exists purely to carry form state between the Activate and Payment pages within one browser tab.

End of document — Vodafone Tourist Pack Platform Technical Documentation.